

Backend Engineer (SME) — Teaching Assignment

Context

A large part of this role is taking a backend concept and turning it into a class that a room of working engineers actually learns from. Not a summary of the topic — a session you could deliver live for two hours.

This assignment is the closest proxy we have to that. We're not testing whether you know these topics; we assume you do. We're testing how you decide to teach them.

What to submit

Two teaching scripts. We will need one script for sure if you have time and bandwidth; then second script creation will be your choice.

Script 1 (required) — Spring Boot: Authentication, Authorisation, OAuth2 and JWT

Script 2 (pick one):

- **Option A** — OOP: Inheritance, Polymorphism, and Why Abstraction Exists
- **Option B** — Java Concurrency: Threads, Race Conditions and Synchronization

Both classes are pitched at **~120 minutes** to an audience of **working software engineers with 0–3 years of experience**. Assume they can write Java and have shipped something small but have never been taught these topics properly.

Topic areas for each are listed at the end of this document.

What we mean by a "script"

A script is what you'd have open in front of you while teaching. Closer to a transcript than to slides or notes.

It should let someone else pick it up and deliver the class. That usually means it contains:

- **Actual spoken language** — the words you would say, not bullet points you would expand from
- **Time markers** — roughly where you expect to be at each stage of the 120 minutes
- **Questions you ask the class and** the answers you expect back (including the wrong ones) and answers.
- **What goes on the board or screen** — diagrams, code, definitions
- **Code you write live**, with the order you'd build it up in
- **The points where learners get stuck and** how you handle it when they do

Format and structure are up to you. Markdown, Google Doc, whatever you'd actually use. There is no template.

Constraints

You decide the coverage. The topic areas below are a pool, not a syllabus. Cutting something and explaining why is a valid choice — arguably a better one than covering everything shallowly. Two hours is less time than it sounds.

Motivate before you explain. A class that opens with "JWT stands for JSON Web Token, and it has three parts" has skipped the only part that's hard to teach. Why does this problem exist? What breaks without this? Get us to want the answer before you give it.

Assume nothing is obvious. Anywhere you catch yourself writing "as you know" or "obviously", that's usually the thing that needs teaching.

On using AI

Use whatever tooling you normally use — Cursor, Claude, Copilot, Codex. We expect it; it's in the job description.

How we'll evaluate this

Roughly in order of weight:

1. **Sequencing** — does concept B land because concept A was set up properly, or is the script a list of topics in arbitrary order?

2. **Depth of explanation** — do you explain *why* something works the way it does, or restate *that* it does?
 3. **Handling of misconceptions** — do you know what learners get wrong here, and do you pre-empt it?
 4. **Technical correctness** — including the code, the edge cases, and the things you choose to simplify (simplifying is fine; being wrong is not)
 5. **Judgement about scope** — what you cut, and whether you can defend the cut
 6. **Clarity of writing** — this role writes a lot of content; the script is a writing sample
-

Topic areas — Script 1 (required)

Spring Boot: Authentication, Authorisation, OAuth2 and JWT

Topic areas — Script 2, Option A

OOP: Inheritance, Polymorphism, and Why Abstraction Exists

Topic areas — Script 2, Option B

Java Concurrency: Threads, Race Conditions and Synchronization

Questions about scope or format are welcome — feel free to reach out to shashank.bose01@scaler.com.